МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

# Отчет по курсовой работе по дисциплине "Функциональное программирование" Реализация проекта на языке Haskell по теме "Визуализатор маршрутов"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

# Содержание

**Техническое задание** **3**

**Введение** **11**

**1 Постановка задач** **12**

**2 Особенности реализации** **13**
2.1 Описание разработки проекта . . . . . 13
2.1.1 Структура проекта . . . . . 13
2.1.2 Диаграмма модулей . . . . . 14
2.1.3 Типы данных . . . . . 14
2.1.4 Стандартные и внешние библиотеки . . . . . 16
2.2 Описание технических особенностей реализации проекта . . . . . 18

**3 Примеры работы программы** **19**

**4 Руководство по эксплуатации для пользователя** **20**

**Заключение** **21**

**Список литературы** **22**

**Приложение** **23**
A.1 . . . . . 23

# Техническое задание

# 1. Основание для выполнения работы

Работа выполняется в соответствии с учебным планом по дисциплине «Функциональное программирование» в рамках курсового проектирования студента группы 5130201/40001 Института компьютерных наук и кибербезопасности Федерального государственного автономного образовательного учреждения высшего образования «Санкт-Петербургский политехнический университет Петра Великого», обучающегося по направлению подготовки 02.03.01 Математика и компьютерные науки.

# 2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40001
Соисполнители: отсутствуют

# 3. Цели, задачи и исходные данные

## a. Цель работы

Целью работы является разработка наглядного инструмента для поиска и визуализации кратчайших маршрутов в транспортных сетях (метрополитен, железные дороги).

## b. Задачи работы

В процессе выполнения работы требуется решить следующие задачи:

- **Чтение и запись файлов:**
  - реализовать импорт графа из JSON-файла, описывающего схему метро/ЖД с использованием библиотеки `aeson`;
  - реализовать экспорт графа в JSON-файл;
  - реализовать сохранение визуализаций графа и найденного кратчайшего пути в PNG-файлы.
- **Обработка информации математическими функциями:**
  - разработать структуры данных для представления взвешенных графов (ориентированных/неориентированных);

  - реализовать алгоритм Дейкстры для поиска кратчайшего пути;
  - реализовать восстановление маршрута от начальной до конечной вершины.

- **Взаимодействие с пользователем через консоль:**
  - разработать систему меню для управления программой;
  - обеспечить ввод начальной и конечной станций для построения маршрута.

- **Логирование действий пользователя:**
  - реализовать запись в лог-файл всех значимых действий;
  - добавить временные метки и уровни логирования.

- **Обработка ошибок:**
  - обеспечить корректную обработку ошибок ввода/вывода;
  - обрабатывать некорректный пользовательский ввод;
  - обрабатывать ситуации, когда путь между станциями отсутствует.

## c. Исходные данные

Исходными данными для проведения работы являются:

- JSON-файлы с описанием графов транспортных сетей в формате, включающем названия станций и перегоны с весами;
- готовые датасеты транспортных сетей (например, из открытых источников);

# 4. Основное содержание работы

## a. Разработка проекта

Разработка проекта включает:

- проектирование архитектуры и системы типов;
- реализацию модулей для работы с графами, алгоритмом Дейкстры, визуализацией, вводом/выводом JSON, логированием и обработкой ошибок;
- написание тестов с использованием QuickCheck;
- отладку и тестирование программы.

## b. Отчетная документация

Отчёт и программа на языке Haskell.

# 5. Основные требования к выполнению работы

## a. Назначение разрабатываемого программного комплекса

Программный комплекс предназначен для поиска и визуализации кратчайших маршрутов в транспортных сетях (метро, ЖД). В зависимости от выбора пользователя оценивает время прохождения или длину маршрута по транспортным сетям. Может использоваться как вспомогательный инструмент для планирования поездок.

## b. Состав разрабатываемого программного комплекса

- Система сборки: Stack;
- JSON-файлы с примерами схем метро и ЖД;
- документация пользователя.

## c. Технические характеристики

**Файлы, используемые программой:**

- JSON-файл с описанием транспортной сети (станция, линия, координаты); Например:

```
{
 "lines": [
 {
  "hex_color": "CD0505",
  "stations": [
  {
   "lat": 53.20969,
   "lng": 50.13442,
   "order": 0,
   "name": "Алабинская",
```

```
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113838,
 "lng": 50.15022,
 "order": 1,
 "name": "Российская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2037926,
 "lng": 50.159832,
 "order": 2,
 "name": "Московская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2004433,
 "lng": 50.1765938,
 "order": 3,
 "name": "Гагаринская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2011223,
 "lng": 50.1992874,
 "order": 4,
 "name": "Спортивная",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2017365,
 "lng": 50.2206573,
```

```
 "order": 5,
 "name": "Советская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2073135,
 "lng": 50.2364159,
 "order": 6,
 "name": "Победа",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2129974,
 "lng": 50.24889,
 "order": 7,
 "name": "Безымянка",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113533,
 "lng": 50.2697754,
 "order": 8,
 "name": "Кировская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2127,
 "lng": 50.28298,
 "order": 9,
 "name": "Юнгородок",
 "admArea": null,
 "district": null,
 "status": null
}
],
```

```
    "id": "54",
    "name": "Первая"
   }
   ],
   "name": "Самара"
  }
```

Исходные данные получены из репозитория: `https://github.com/morphey83/MetroAndDistrict/blob/master/metro.all.json` (Дата обращения 03.04.2026)

- JSON-файл для экспорта текущей схемы;
- PNG-файл визуализации всей схемы;
- PNG-файл визуализации найденного маршрута;
- текстовый файл лога с временными метками и уровнями логирования.

**Что технически может сделать пользователь:**

- Загрузить схему из JSON-файла;
- Экспортировать текущую схему в JSON-файл;
- Просмотреть список всех станций;
- Найти станцию по части названия;
- Выбрать начальную и конечную станцию;
- Запустить алгоритм Дейкстры для поиска кратчайшего пути;
- Просмотреть маршрут и его длину;
- Сохранить визуализацию всей схемы в PNG;
- Сохранить визуализацию маршрута с выделенным путём в PNG;
- Просмотреть последние записи лога или весь лог;
- Очистить лог-файл.

# d. Функциональные требования

## d.1. Требования к производительности

- Загрузка и парсинг JSON-файла — не более 2 секунд.
- Работа алгоритма Дейкстры для графа до 500 вершин — не более 0.5 секунды.

- Генерация PNG-визуализации — не более 3 секунд.
- Потребление оперативной памяти — не более 200 МБ.

### d.2. Требования к точности и обработке данных

- Алгоритм Дейкстры должен находить кратчайший путь с точностью 100%.
- Веса рёбер поддерживаются дробными (точность до 0.1).
- При импорте JSON выполняется проверка ссылочной целостности.
- Программа корректно обрабатывает неориентированные графы.

### d.3. Требования к хранению и логированию

- Все действия пользователя фиксируются в лог-файле.
- Максимальный размер лога — 10 МБ, при превышении старая запись перезаписывается.
- Экспорт сохраняет полную структуру графа без потерь.

### d.4. Требования к надёжности

- Программа не должна завершаться аварийно при некорректном вводе.
- При отсутствии пути между станциями выводится сообщение об ошибке.
- Визуализация невозможна без загруженной схемы.

## e. Стандартные средства

При разработке должны использоваться типовые средства языка Haskell и существующие библиотечные решения:

- `aeson` — для парсинга и генерации JSON-файлов;
- `containers` (Data.Map, Data.Set, Data.Sequence) — для представления графов и структур данных алгоритмов;
- `graphite` — для визуализации графов и сохранения в PNG;
- `QuickCheck` — для тестирования свойств;
- `directory` — для работы с файловой системой;
- `time` — для формирования временных меток в логах;
- `bytestring` — для эффективной работы с JSON.

# 6. Этапы и сроки выполнения работы

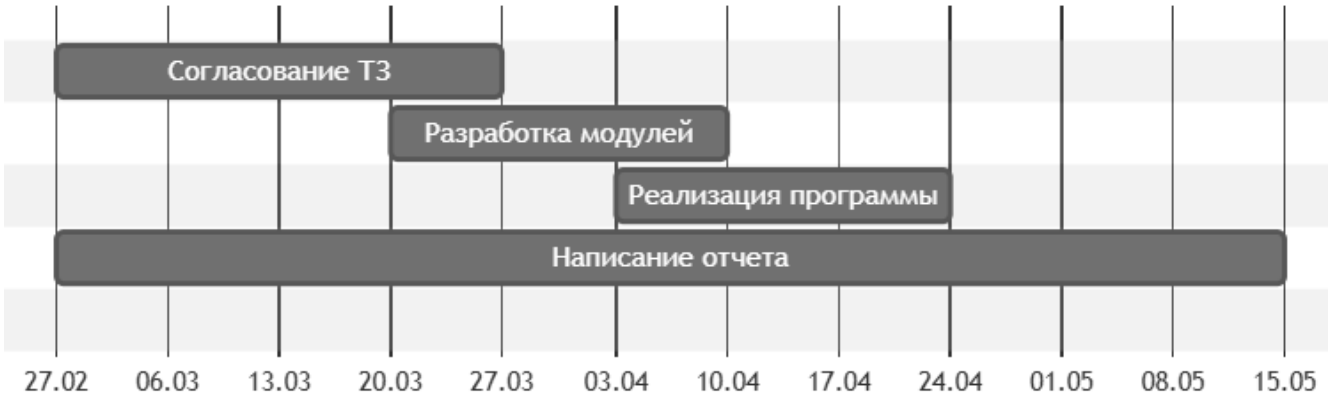



Рис. 1: Диаграмма Ганта этапов работ

# 7. Результаты работы

1. Утверждённое техническое задание.
2. Диаграмма модулей, описание типов данных, обоснование выбора библиотек.
3. Работающие модули парсинга JSON, поиска маршрута (алгоритм Дейкстры), визуализации, консольного интерфейса, логирования и обработки ошибок.
4. Готовый отчёт с описанием разработки, примерами работы и руководством пользователя.

# Введение

Сейчас транспортные системы становятся всё сложнее: метро, железные дороги и их пересечения образуют разветвлённые сети. Поэтому умение быстро находить оптимальный маршрут от точки А до точки Б важно не только для разработчиков навигаторов, но и для многих IT-специалистов, работающих с данными, визуализацией или логистикой.

Для моделирования таких сетей удобно использовать графы [1]. В этой модели станции становятся вершинами, перегоны между ними — рёбрами, а веса рёбер могут означать время поездки или расстояние. Классическим алгоритмом поиска кратчайшего пути во взвешенном графе является алгоритм Дейкстры [2]. Понимание того, как он работает и какие структуры данных (например, очереди с приоритетом) нужны для его эффективной реализации, помогает правильно решать задачу маршрутизации.

Помимо самого поиска пути, важна практическая работа с данными. Схемы транспортных сетей удобно хранить в формате JSON [3], а для их парсинга в Haskell использовать библиотеку `aeson` [4]. Также важна визуализация: найденный маршрут должен быть не просто списком станций, но и наглядной картинкой. Для этого подходит библиотека `graphite` [5], которая позволяет строить схему по координатам станций и сохранять результат в PNG.

# 1 Постановка задач

# 2 Особенности реализации

## 2.1 Описание разработки проекта

### 2.1.1 Структура проекта

```
route-visualizer/
  app/
    Main.hs               # Точка входа, консольное меню
  src/
    Graph/
      Dijkstra.hs         # Алгоритм Дейкстры
      Utils.hs            # Вспомогательные функции
    IO/
      JSONParser.hs       # Импорт/экспорт JSON (aeson)
      Logger.hs           # Логирование (time, directory)
    Visualization/
      Render.hs           # Визуализация в PNG (graphite)
    CLI/
      Menu.hs             # Консольный интерфейс
  test/
    Spec.hs               # Тесты QuickCheck
  data/
    metro.json            # Входные данные
  logs/
    app.log               # Лог-файл
  output/
    full_graph.png
    route.png
  package.yaml            # Stack конфигурация
  stack.yaml
```

### 2.1.2 Диаграмма модулей

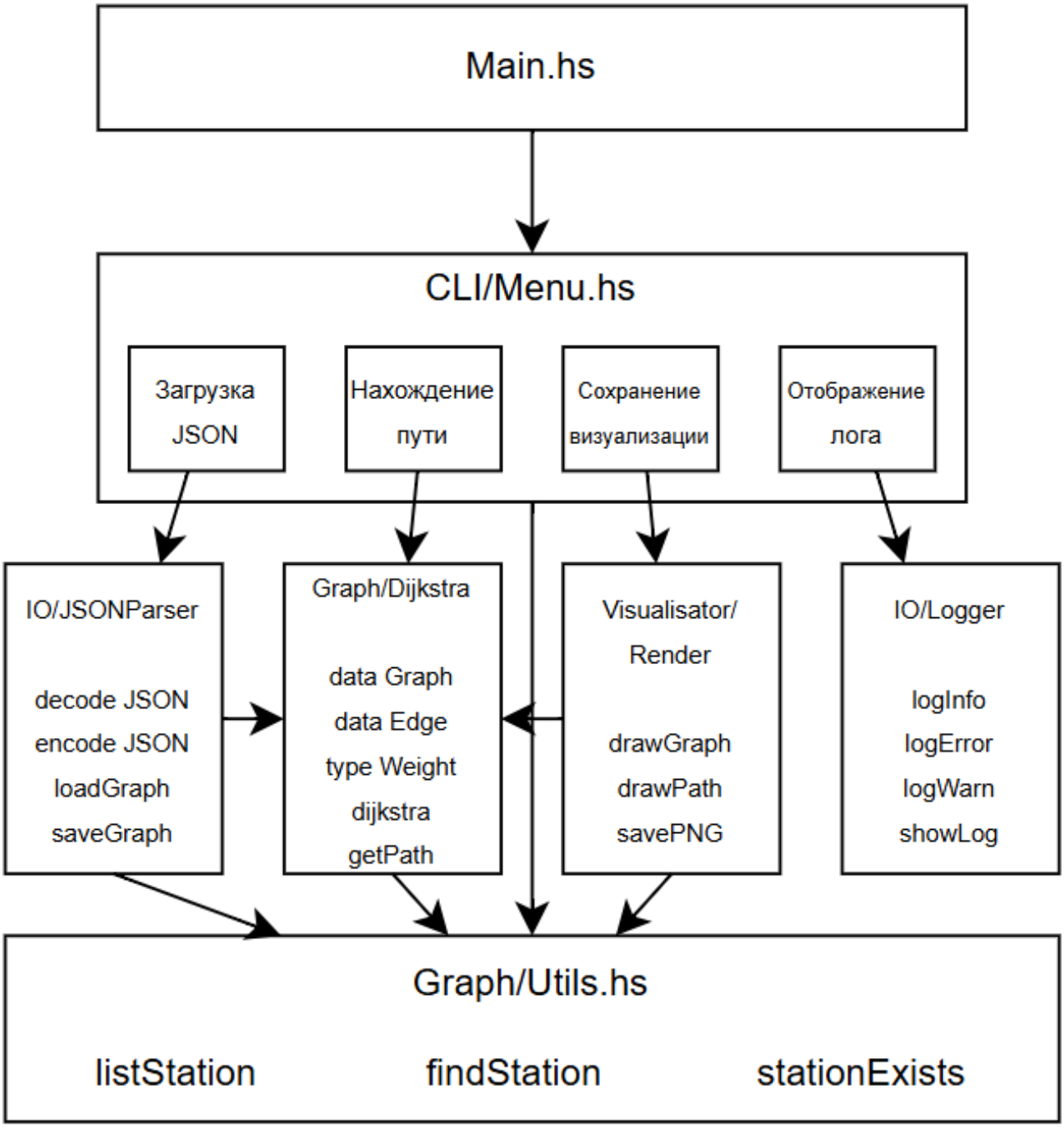



Рис. 2: Диаграмма модулей

### 2.1.3 Типы данных

Все основные типы данных определены в модуле `Graph/Dijkstra.hs`. Ниже приведено их описание.

#### Базовые типы

- `type Weight = Double` — вес ребра (расстояние в километрах). Используется `Double` для поддержки дробных значений.
- `type StationName = String` — название станции, уникальный идентификатор вершины графа.
- `type Coordinates = (Double, Double)` — координаты станции (широта, долгота) для визуализации.
- `type Edge = (StationName, StationName, Weight)` — ребро графа: станция отправления, станция назначения, вес перегона.

## Граф транспортной сети

`data Graph = Graph { graphStations :: [StationName], graphEdges :: [Edge], graphCoordinates :: Map StationName Coordinates }` — транспортный граф. Хранит все станции, связи между ними и координаты каждой станции. Deriving: `Show`, `Eq`.

## Результат работы алгоритма Дейкстры

`type RouteResult = Maybe ([StationName], Weight)` — результат поиска маршрута. `Nothing` означает, что путь не найден. `Just (путь, общий вес)` содержит последовательность станций и суммарный вес.

## Типы для логирования

- `data LogLevel = Info | Warning | Error` — уровень важности сообщения в логе. `Info` — обычное действие пользователя, `Warning` — некритичная проблема, `Error` — ошибка (файл не найден, путь не существует). Deriving: `Show`, `Eq`.
- `data LogEntry = LogEntry { logTimestamp :: UTCTime, logLevel :: LogLevel, logMessage :: String }` — одна запись в лог-файле. Содержит временную метку, уровень важности и текст сообщения. Deriving: `Show`, `Eq`.

## Типы для работы с JSON (импорт/экспорт)

- `data MetroJSON = MetroJSON { name :: String, lines :: [Line] }` — структура JSON-файла. Содержит название города (поле `name`) и список линий метро (поле `lines`). Deriving: `Show`, `Eq`.
- `data Line = Line { lineHexColor :: String, lineStations :: [Station] }` — одна линия метро. Содержит цвет линии в HEX (например, `"#CD0505"`) и список станций. Deriving: `Show`, `Eq`.

- `data Station = Station { stationName :: String, stationLat :: Double, stationLng :: Double, stationOrder :: Int }` — одна станция внутри линии. Содержит название, широту, долготу и порядковый номер на линии (0, 1, 2, ...). Deriving: `Show`, `Eq`.

### Типы для обработки ошибок

- `data AppError = FileNotFound FilePath | JsonParseError String | StationNotFound StationName | PathNotFound StationName StationName | GraphNotLoaded` — ошибки, которые могут возникнуть при работе программы. Deriving: `Show`, `Eq`.
- `type AppResult a = Either AppError a` — результат операции с явной обработкой ошибки. Используется в модулях `JSONParser` и других.

## 2.1.4 Стандартные и внешние библиотеки

### Внешние библиотеки

- `aeson` — библиотека для парсинга и генерации JSON. Используются функции:
  - `decode :: FromJSON a => ByteString -> Maybe a` — преобразует JSON-строку в Haskell-значение типа, реализующего класс `FromJSON`. В проекте применяется для загрузки транспортных схем из файлов.
  - `encode :: ToJSON a => a -> ByteString` — выполняет обратное преобразование, необходимое для экспорта графа в JSON.

  **Обоснование:** `aeson` — стандарт в Haskell для работы с JSON, обеспечивает типобезопасность и высокую производительность.
- `containers` — набор эффективных структур данных. Используются:
  - `Map` — для хранения координат станций (`graphCoordinates`) с быстрым доступом по ключу (названию станции).
  - `Set` — для отслеживания посещённых вершин в алгоритме Дейкстры.

  **Обоснование:** Эти структуры оптимальны для алгоритмов на графах. `Map` обеспечивает логарифмическое время доступа к координатам, что критично для визуализации. `Set` позволяет быстро проверять, была ли уже обработана вершина.
- `graphite` — библиотека для визуализации графов. Используются функции:
  - `renderGraph` — создаёт изображение графа по заданным вершинам, рёбрам и координатам.

  - `savePng` — сохраняет полученное изображение в PNG-файл.
  - `drawPath` — выделяет заданный маршрут (например, красным цветом) поверх основной схемы.

  **Обоснование:** `graphite` предоставляет инструмент для визуализации графов по координатам вершин. Функция `drawPath` позволяет наглядно показать найденный маршрут без ручного рисования каждого ребра.

- `QuickCheck` — библиотека для property-based тестирования. Используются:
  - `property` — для формулировки свойств, которые должны выполняться для любых входных данных (например, *«найденный путь не длиннее любого другого пути между теми же вершинами»*).
  - `quickCheck` — для запуска тестов.
  - `arbitrary` — для автоматической генерации случайных графов.

  **Обоснование:** `QuickCheck` позволяет проверить корректность алгоритма Дейкстры на тысячах случайных графов, что сложнее и надёжнее, чем ручное написание фиксированных примеров.

- `time` — для работы со временем. Используется `getCurrentTime :: IO UTCTime` для получения текущего времени при записи в лог.
  **Обоснование:** даёт точную временную метку, необходимую для логирования порядка действий пользователя.

- `directory` — для работы с файловой системой. Используются функции:
  - `doesFileExist` — проверка существования файла перед чтением (чтобы избежать ошибок).
  - `createDirectoryIfMissing` — автоматическое создание директорий `logs/` и `output/` при первом запуске.

  **Обоснование:** способ работы с файловой системой, позволяет избежать падений программы из-за отсутствия директорий.

## Стандартные модули GHC

- `Prelude` — базовый модуль, предоставляющий основные функции: `map`, `filter`, `foldl`, `maybe`, `either`, операторы работы со списками.
- `Data.Maybe` — для работы с значениями типа `Maybe`: `fromMaybe` (значение по умолчанию), `catMaybes` (фильтрация пустых значений).
- `Data.Either` — для работы с `Either`: `lefts`, `rights`, `isLeft`, `isRight`. Используется при обработке ошибок в `AppResult`.

- `System.IO` — для консольного ввода-вывода: `hPutStrLn`, `stderr`, `stdout`, `getLine`.
- `Control.Exception` — для перехвата исключений: `try`, `catch`, `SomeException`. Используется для обработки ошибок ввода-вывода (например, файл не найден).

## 2.2 Описание технических особенностей реализации проекта

# 3 Примеры работы программы

# 4 Руководство по эксплуатации для пользователя

# Заключение

# Список литературы


[1] Новиков Ф.А. Дискретная математика для программистов. 3-е изд. – Санкт-Петербург : Питер Пресс, 2009. – 384 с.

[2] Левитин А. В. Алгоритмы: введение в разработку и анализ. – М. : Вильямс, 2006. – 576 с.

[3] RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. – URL: `https://datatracker.ietf.org/doc/html/rfc8259` (дата обращения: 09.04.2026).

[4] Hackage: aeson – A JSON parsing and encoding library for Haskell. – URL: `https://hackage.haskell.org/package/aeson` (дата обращения: 09.04.2026).

[5] Hackage: graphite – A library for rendering graphs and exporting to PNG. – URL: `https://hackage.haskell.org/package/graphite` (дата обращения: 09.04.2026).

# Приложение

## A.1